
LECTURE NOTES

Lecture 15: Discrete Optimization

University

Simon Holm
April - 2026



DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

1	Dynamic Programming	3
2	Overlapping subproblems	3
3	Optimal substructure	3
4	Constraint programming	5
4.1	The core of constraint computation	5

1 Dynamic Programming

This is applied to problem with **optimal substructure** and **overlapping subproblems**

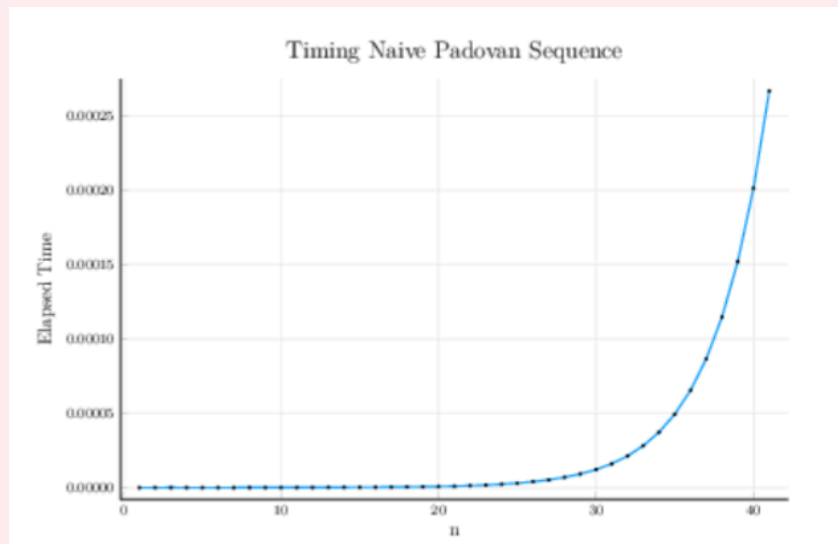
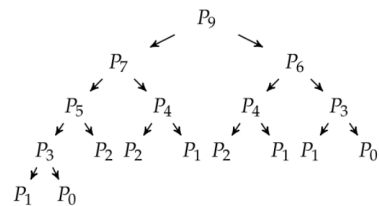
2 Overlapping subproblems

A problem has overlapping subproblems when solving it recursively involves solving the same subproblems multiple times rather than always generating new ones.

Example: padovan_naive

$$P_n = P_{n-2} + P_{n-3} \quad P_0 = P_1 = P_2 = 1$$

```
def padovan_naive(n, P=None):  
    if n < 3:  
        return 1  
    else:  
        return padovan_naive(n - 2, P) ↪  
               ↪ + padovan_naive(n - 3, ↪  
               ↪ P)
```



Notice here that some computations are done multiple times

of overlapping subproblems

3 Optimal substructure

The **Principle of Optimality** (known as Bellman Optimality Conditions):

A problem has optimal substructure if an optimal solution to the whole problem contains optimal solutions to its subproblems.

Example

If the shortest path from A to D goes $A \rightarrow B \rightarrow C \rightarrow D$, then:

- $B \rightarrow C \rightarrow D$ must be the shortest path from B to D
- $C \rightarrow D$ must be the shortest path from C to D

Example 2: The knapsack problem

Capacity = 5 kg

Item	Weight	Value
1	1 kg	\$1
2	3 kg	\$4
3	4 kg	\$5

Step 1: Initialize

Create a table with (number of items + 1) rows and (capacity + 1) columns, filled with zeros.

```
w = 0 1 2 3 4 5
i=0 | 0 0 0 0 0 0
i=1 | 0
i=2 | 0
i=3 | 0
```

Step 2: Fill it step by step

Item 1 ($w_1=1, v_1=1$):

```
w = 0 1 2 3 4 5
i=0 | 0 0 0 0 0 0
i=1 | 0 1 1 1 1 1
i=2 | 0
i=3 | 0
```

Item 2 ($w_2=3, v_2=4$):

```
w = 0 1 2 3 4 5
i=0 | 0 0 0 0 0 0
i=1 | 0 1 1 1 1 1
i=2 | 0 1 1 4 5 5
i=3 | 0
```

Item 3 ($w_3=4, v_3=5$):

```
w = 0 1 2 3 4 5
i=0 | 0 0 0 0 0 0
i=1 | 0 1 1 1 1 1
i=2 | 0 1 1 4 5 5
i=3 | 0 1 1 4 5 6
```

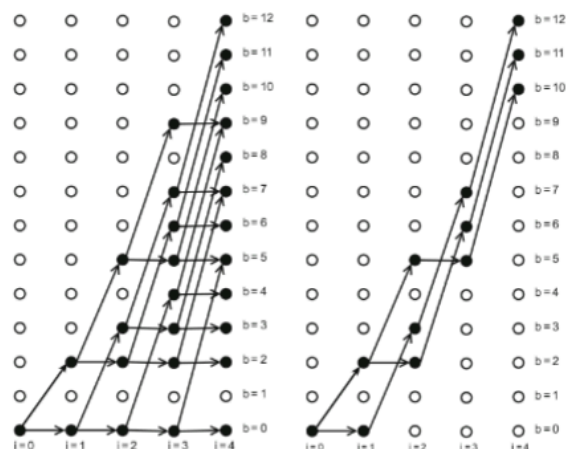
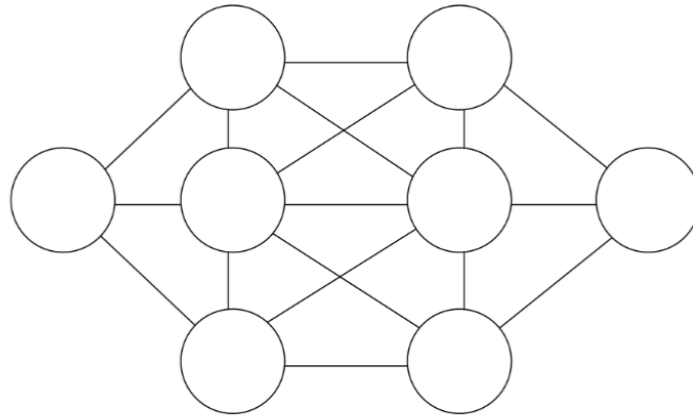


Figure 4:

4 Constraint programming

Example: Number Circle Puzzle



Put a different number (1 to 8) in each circle such that adjacent circles do not have consecutive numbers.

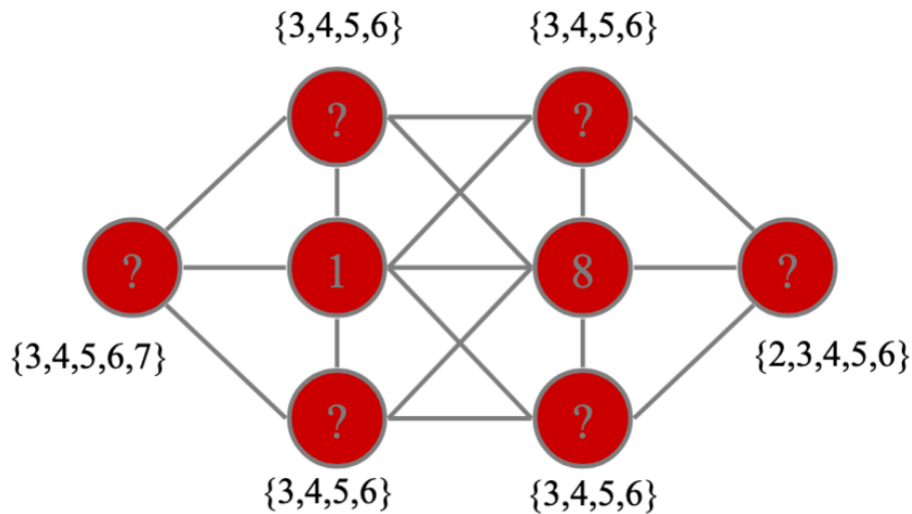
Note the symmetry in the this problem

There are many parts to solving problems like this.

4.1 The core of constraint computation

Modelling (deciding on variables/domains/constraints)

- Inference/propagation
- Heuristics
- Symmetry
- Backtracking



Definition of Variables, Domains, Constraints

The **domain** of a variable x , denoted $D(x)$, is a finite set of elements that can be assigned to x .

A **constraint** C on X is a subset of the Cartesian product of the domains of the variables in X , *i.e.*, $C \subseteq D(x_1) \times \dots \times D(x_k)$. A tuple $(d_1, \dots, d_k) \in C$ is called a solution to C .

Equivalently, we say that a solution $(d_1, \dots, d_k) \in C$ is an assignment of the value d_i to the variable x_i for all $1 \leq i \leq k$, and that this assignment satisfies C .

If $C = \emptyset$, we say that it is **inconsistent**.

Extensional: specifies the good (or bad) tuples (values) **Intensional**: specifies the characteristic function

Definition: Constraint Satisfaction Problem (CSP)

A CSP is a finite set of variables X with domain extension $D = D(x_1) \times \dots \times D(x_n)$, together with a finite set of constraints C , each on a subset of X . A solution to a CSP is an assignment of a value $d \in D(x)$ to each $x \in X$, such that all constraints are satisfied simultaneously.

Definition: Constraint Optimization Problem (COP)

A COP is a CSP $\mathcal{P}$ defined on the variables $x_1, \dots, x_n$, together with an objective function $f : D(x_1) \times \dots \times D(x_n) \rightarrow Q$ that assigns a value to each assignment of values to the variables. An optimal solution to a minimization (maximization) COP is a solution d to P that minimizes (maximizes) the value of $f(d)$.